



RESUMEN PRIMER PARCIAL



Este resumen está pensado para **resolver preguntas del primer parcial**. Prioriza el material de la cátedra y las correcciones reales que fuimos revisando. Cuando una pregunta venga de un parcial corregido, la clave de la docente tiene prioridad para practicar ese examen.



UNIDAD 1 — Ingeniería de Software y Gestión de Proyectos



1. Ingeniería de Software: qué es y por qué existe

La Ingeniería de Software es la disciplina que se ocupa de **todos los aspectos de la producción del software**, desde la especificación inicial hasta el mantenimiento después de su puesta en operación.

No es simplemente programar. Incluye comprender el problema, relevar necesidades, diseñar, implementar, probar, desplegar, documentar, gestionar el proyecto y mantener el producto.

Idea clave para preguntas: antes de pensar una solución hay que comprender el problema y las necesidades del cliente. Un buen diseño mejora la calidad y la mantenibilidad.

▼ Disciplinas principales

- **Técnicas:** requerimientos, análisis, diseño, implementación, testing, despliegue, documentación y capacitación.
- **Gestión:** planificación, monitoreo y control del proyecto.

- **Soporte:** actividades transversales que preservan integridad y calidad, como SCM, métricas y QA.

▼ Software vs. manufactura

No conviene comparar directamente el desarrollo de software con una línea de producción industrial.

- El software es **menos predecible**.
- No existe producción en masa de productos idénticos en el mismo sentido que en manufactura.
- No todas las fallas son simplemente "errores de fabricación".
- El software **no se desgasta físicamente** con el uso.
- No está gobernado por leyes físicas de la misma manera que un producto material.
- Depende fuertemente de **personas, conocimiento, contexto y creatividad**.

Trampa típica: "Si repito exactamente el mismo proceso en otro proyecto voy a obtener el mismo resultado". → **No necesariamente.**

▼ 2. Crisis del software, éxitos y fracasos

La crisis del software surge cuando la complejidad de los sistemas crece más rápido que nuestra capacidad para planificarlos y construirlos correctamente.

Problemas típicos:

- retrasos;
- sobrecostos;
- baja calidad;
- poca mantenibilidad;
- producto que no cumple los requisitos del cliente.

▼ Factores de éxito

- monitorear el proyecto;
- recibir feedback;

- comunicación fuerte entre actores;
- misión, visión y objetivos claros;
- seguimiento real del plan cuando corresponde.

▼ Causas frecuentes de fracaso

- planificar con datos insuficientes;
- no hacer seguimiento del proyecto;
- mala gestión de recursos;
- estimar sin datos históricos;
- requerimientos mal relevados;
- plan de proyecto pobre en detalles;
- responsabilidades poco claras.

Regla de examen: un proyecto puede “atrasarse de a un día por vez” porque nadie compara periódicamente la realidad contra el plan.



3. Ciclos de vida: secuencial, iterativo-incremental y recursivo

Un ciclo de vida es una representación simplificada del proceso de desarrollo. Define **qué fases existen, en qué orden se realizan y qué condiciones permiten avanzar**.

▼ Secuencial / tradicional

Pensalo como: **certeza + plan + documentación**.

- Las fases se realizan de manera lineal.
- Una etapa debe completarse antes de comenzar la siguiente.
- Funciona mejor cuando los requerimientos son claros, exhaustivos y estables.
- El cliente suele ver resultados funcionales tarde.
- Los cambios tardíos son costosos.

- El seguimiento se realiza comparando el avance real contra el plan.

Palabras que lo delatan: cronograma detallado, horas/días, plan base, revisión formal, control de cambios, Gantt, requisitos estables.

▼ Iterativo-incremental

Pensalo como: **cambio + entregas parciales + feedback.**

- El producto crece mediante sucesivas iteraciones.
- Cada incremento agrega funcionalidad.
- Permite entregar valor antes de terminar todo el producto.
- Facilita incorporar requisitos cambiantes.
- El cliente participa y puede dar feedback temprano.
- Es el enfoque normalmente asociado a metodologías ágiles.

▼ Recursivo / espiral

Pensalo como: **riesgo.**

Cada vuelta suele seguir esta lógica:

1. determinar objetivos;
2. identificar y resolver riesgos;
3. desarrollar y probar;
4. planificar la siguiente iteración.

Se utiliza especialmente en sistemas grandes y complejos donde interesa reducir riesgos progresivamente.

▼ Machete de elección

- **Requisitos claros + baja incertidumbre → Secuencial.**
- **Requisitos variables + entregas parciales → Iterativo-incremental.**
- **Proyecto grande + riesgos altos → Recursivo/Espiral.**

▼ 4. Procesos definidos vs. empíricos

▼ Procesos definidos

Buscan **predictibilidad y repetibilidad**. Suponen que si un proceso se ejecuta de forma similar, debería producir resultados previsibles.

Se asocian bien con contextos de mayor certeza, alcance conocido y planificación previa.

▼ Procesos empíricos

Aceptan que el desarrollo de software es complejo y contextual, por lo que **el mismo proceso puede producir resultados diferentes**.

Se controlan mediante ciclos cortos de aprendizaje y tres pilares:

- **Transparencia:** hacer visible el estado real del trabajo.
- **Inspección:** revisar frecuentemente resultados y progreso.
- **Adaptación:** ajustar el trabajo según lo aprendido.

Trampa típica: obtener resultados distintos al repetir prácticas **no invalida** un proceso empírico; es algo esperable en contextos complejos.

▼ 5. Criterios para elegir un ciclo de vida

La elección depende de:

- nivel de riesgo;
- claridad y estabilidad de requerimientos;
- conocimiento técnico del equipo;
- plazos disponibles;
- posibilidad de validar resultados con el cliente;
- necesidad o no de entregas parciales.

No existe un ciclo de vida universalmente mejor: se selecciona según el contexto del proyecto y del producto.

▼ 6. Proceso, proyecto y producto

- **Proceso:** conjunto de actividades, métodos, prácticas y transformaciones que indica **cómo trabajar**.
- **Proyecto:** esfuerzo temporal que organiza personas y recursos para obtener un resultado único.
- **Producto:** resultado de software que puede continuar evolucionando mucho después de que termine un proyecto.

▼ Ciclo de vida del proyecto vs. producto

El ciclo del **proyecto** termina cuando el esfuerzo temporal alcanza su objetivo.

El ciclo del **producto** puede continuar durante años e incluir varios proyectos: nuevas versiones, ampliaciones, migraciones, etc.

Relación: el proceso se adapta al proyecto; el proyecto incorpora personas y herramientas; del proyecto surge o evoluciona un producto.

▼ 7. Plan de proyecto y gestión tradicional

El Plan de Proyecto funciona como una **hoja de ruta**. Debe dejar claro:

1. qué se hace — alcance;
2. cómo se hace — actividades y tareas;
3. qué riesgos existen;
4. quién lo hace — responsables;
5. cuándo se hace — calendario;
6. cómo se controla — métricas y mecanismos de seguimiento.

▼ Qué reconocer en preguntas de “gestión tradicional”

- descomposición del trabajo en tareas;
- cronograma con fechas;
- estimaciones en horas/días;
- datos históricos;

- supuestos documentados;
- contingencias y riesgos;
- comparación **plan vs. real**;
- cambios gestionados formalmente.

Incorrecto: cambiar alcance, cronograma o configuración "sin registrarlo para evitar burocracia".

▼ 8. Triple restricción: alcance, tiempo y costo

Las tres restricciones principales son **alcance, tiempo y costo**. Están relacionadas y su equilibrio impacta en la calidad.

- **Alcance:** qué funcionalidades y trabajo deben realizarse.
- **Tiempo:** cuándo debe estar terminado.
- **Costo:** recursos necesarios para realizarlo.

▼ En enfoque tradicional

Se intenta fijar el alcance y a partir de él estimar tiempo y costo.

Si aumenta el alcance, normalmente deben aumentar tiempo y/o costo. Pretender mantener todo fijo puede deteriorar la calidad.

▼ En enfoque ágil

La lógica suele invertirse: se busca trabajar con **equipo/recursos relativamente estables y timeboxes fijos**, mientras el alcance se adapta según prioridades y feedback.

Regla de examen: para mantener presupuesto y plazo, se prioriza valor y se negocia alcance; no se "elimina" la restricción de costo.

▼ 9. Estimaciones tradicionales

En un enfoque tradicional se intenta predecir tamaño, esfuerzo, calendario y costo con mayor detalle desde etapas tempranas.

Buenas prácticas que aparecieron en los parciales:

- descomponer el trabajo en tareas pequeñas;
- utilizar información histórica de proyectos similares;

- documentar supuestos;
- considerar disponibilidad de recursos y calendario;
- incluir contingencias para riesgos;
- no ignorar actividades intermedias.

▼ Diferencias importantes

- **Tamaño:** magnitud del producto.
- **Esfuerzo:** horas-persona necesarias.
- **Calendario:** distribución del trabajo en fechas reales.
- **Costo:** recursos económicos necesarios.

No son sinónimos: un trabajo puede requerir cierto esfuerzo pero terminarse en una fecha distinta según disponibilidad, dependencias y contexto.

▼ 10. No Silver Bullet — Fred Brooks

Brooks plantea que **no existe una herramienta o tecnología mágica** capaz de eliminar las dificultades fundamentales del desarrollo de software.

▼ Dificultades accidentales

Surgen de las herramientas o mecanismos de implementación. Pueden reducirse con mejores lenguajes, IDEs, automatización, IA, frameworks, etc.

▼ Dificultades esenciales

Son inherentes al problema de software:

- **Complejidad;**
- **Conformidad** con reglas y entornos externos;
- **Cambiabilidad/Mutabilidad;**
- **Invisibilidad.**

Trampa típica: "IA/Jira/Slack/low-code va a resolver el problema". → Puede ayudar con aspectos accidentales, **pero no elimina las**



UNIDAD 2 — Gestión de Producto, Requerimientos Ágiles, US, Estimación y Scrum

▼ 1. Gestión de Producto: valor y experimentación

El desarrollo de producto debe concentrarse en **crear valor para el usuario y el negocio**, evitando construir grandes cantidades de funcionalidad que luego nadie utiliza.

▼ MVP — Minimum Viable Product

El MVP se orienta principalmente al **aprendizaje validado**.

Es la versión de un producto que permite probar hipótesis reales con clientes utilizando el menor esfuerzo razonable. No significa “producto malo”: debe ser suficientemente viable como para que el feedback sea válido.

Puede validar:

- **Hipótesis de valor:** ¿el producto realmente aporta valor?
- **Hipótesis de crecimiento:** ¿cómo llegarán nuevos usuarios?

Idea clave: observar lo que el usuario realmente hace suele ser más confiable que preguntarle qué haría.

▼ MVF, MMF, MMP y MMR

- **MVF — Minimum Viable Feature:** versión mínima de una característica para probar utilidad y adopción.
- **MMF — Minimum Marketable Feature:** característica mínima suficientemente completa como para generar valor comercial.
- **MMP — Minimum Marketable Product:** producto mínimo suficientemente completo para ser comercializable y competitivo para primeros usuarios.

- **MMR — Minimum Marketable Release:** release mínimo que agrega valor comercial; en el material se relaciona MMP con el primer MMR.

Machete: MVP = aprender. MMF/MMP/MMR = comercializar/ganar valor de mercado.

▼ MLP — concepto que apareció en preguntas

El MLP apareció en preguntas de práctica como una variante orientada a generar una experiencia que el usuario **quiera/love**, además de cumplir lo mínimo. **No está desarrollado en profundidad en el material cargado**, por lo que conviene recordar solo la asociación usada en los cuestionarios: *MLP → experiencia/deleite emocional*.

▼ Riesgo de construir sin validar

Si se construye una feature completa antes de validar valor, aumentan riesgos como:

- desperdicio;
- sobreingeniería;
- construir algo que nadie quiere;
- perder feedback temprano.

▼ 2. Requerimientos ágiles

En software, uno de los problemas más difíciles es decidir **qué producto construir**. Los requerimientos cambian, los usuarios descubren nuevas necesidades y muchas veces no pueden expresar todo desde el inicio.

Agile responde mediante:

- comunicación continua cliente-equipo;
- entregas tempranas;
- priorización por valor;
- documentación “solo suficiente”;
- adaptación del alcance según nueva información.

▼ Dominio del problema vs. dominio de la solución

- **Dominio del problema:** qué necesita el negocio y el usuario.
- **Dominio de la solución:** cómo el equipo implementará técnicamente esas necesidades.

Las User Stories trabajan principalmente a nivel de **requerimientos de usuario**, no describiendo la solución técnica detallada.

▼ 3. Product Backlog

Es una lista **emergente, ordenada y priorizada** del trabajo necesario para mejorar el producto.

El Product Owner:

- ordena el backlog;
- prioriza según valor;
- puede cambiar prioridades;
- incorpora nuevas necesidades a medida que se descubren.

Trampa típica: "el Product Backlog queda congelado al comenzar el release". → **Incorrecto**. Es evolutivo.

▼ 4. User Stories: qué son y cómo identificarlas

Una User Story es una descripción corta de una funcionalidad esperada por un usuario. Funciona como **recordatorio de una conversación**, no como especificación exhaustiva.

Formato típico:

Como [rol] quiero [acción] para [valor de negocio].

▼ Componentes principales

- **Rol:** quién realiza la acción o recibe el valor.
- **Acción:** funcionalidad concreta que debe soportar el sistema.
- **Valor:** por qué esa funcionalidad importa al negocio/usuario.

▼ Porción vertical

Una buena US entrega una funcionalidad completa atravesando lo necesario de interfaz, lógica y datos.

Si una historia es demasiado grande: dividirla en historias más pequeñas **verticales y con valor**, no simplemente convertirla en “frontend, backend y base de datos”. Eso último son tareas técnicas.



5. Criterios de aceptación, conversación y confirmación

▼ Criterios de aceptación

Definen los límites y condiciones que debe satisfacer la historia.

Deben:

- expresar una intención, no una solución técnica;
- ser independientes de la implementación;
- mantenerse a un nivel relativamente alto;
- facilitar el trabajo de desarrollo y testing.

No sirven para especificar la arquitectura.

▼ Conversación

Es la parte central de la US. PO y equipo desarrollan juntos los detalles necesarios para comprender la necesidad.

Los criterios de aceptación NO reemplazan la conversación.

▼ Confirmación / pruebas de aceptación

Permiten demostrar que la funcionalidad implementada satisface lo acordado. Las pruebas de aceptación derivan de los criterios y de los detalles surgidos en la conversación.

▼ 6. DoR, DoD, INVEST, Épicas y Spikes

▼ Definition of Ready — DoR

Pregunta mental: **¿está lista para empezar?**

Es un criterio de calidad que determina si una US está suficientemente preparada para entrar a una iteración.

Puede considerar claridad, estimación, criterios de aceptación, dependencias y otras condiciones acordadas por equipo y PO.

▼ Definition of Done — DoD

Pregunta mental: **¿está realmente terminada?**

Es un checklist compartido que determina cuándo un incremento/US cumple la calidad requerida y puede considerarse completo.

Machete: DoR = entrar. DoD = salir.

▼ INVEST

Una buena US debería ser:

- Independiente;
- Negociable;
- Valuable;
- Estimable;
- Small;
- Testeable.

▼ Épica

Historia demasiado grande para completarse dentro de una iteración. Debe dividirse en US más pequeñas antes de comprometerla.

▼ Spike

Trabajo especial destinado a **reducir una incertidumbre importante.**

- **Técnico:** investigar tecnología, performance o forma de implementación.

- **Funcional:** resolver dudas sobre interacción o comportamiento esperado del usuario.

Una vez resuelta la incertidumbre, puede derivar en una o más User Stories estimables.

▼ 7. Estimaciones ágiles y Story Points

Una estimación es una **predicción**, no un compromiso ni un plan.

▼ Story Points

Los SP miden **tamaño relativo**, no horas.

Consideran principalmente:

- complejidad;
- esfuerzo;
- incertidumbre.

Una historia de 8 SP no significa "8 horas". Tampoco deben utilizarse para comparar productividad entre equipos diferentes.

▼ US canónica

Se selecciona una historia simple y con poca incertidumbre como referencia. Las demás se estiman comparándolas contra esa historia.

No importa que el número "1", "2" o "3" tenga significado absoluto: lo importante es mantener consistencia relativa dentro del equipo.

▼ Planning Poker

El equipo que realiza el trabajo estima colectivamente.

1. cada miembro elige un valor;
2. todos revelan simultáneamente;
3. se discuten diferencias, especialmente valores extremos;
4. se repite hasta lograr una estimación razonable.

El PO puede aportar **contexto de negocio**, pero la estimación corresponde al equipo que hará el trabajo.

▼ Velocidad

La velocidad **se mide, no se estima**. Es la suma de SP de las historias completadas durante una iteración.

Solo cuentan historias que cumplen DoD y son aceptadas; no se suman "porcentajes" de historias parcialmente terminadas.

Una velocidad estable ayuda a obtener previsibilidad y planificar futuras iteraciones. **No significa que el equipo deba dejar de estimar.**

▼ Reestimación

Puede tener sentido cuando nueva información cambia la percepción relativa del tamaño de una historia. No se reestima simplemente para "forzar" que la velocidad coincida con lo esperado.

▼ Capacidad

Representa cuánto trabajo/tiempo disponible tiene realmente el equipo durante una iteración. Complementa a la velocidad para planificar cuánto puede comprometerse.

▼ WSJF — apareció en preguntas

WSJF = Weighted Shortest Job First. Se usa para priorizar trabajo relacionando valor/costo de demora con tamaño del trabajo.

Para el parcial alcanza con recordar: **priorizar mayor beneficio relativo respecto al tamaño/esfuerzo**. No está desarrollado en profundidad en el material base.

▼ 8. Planificación ágil: Release, Iteración y Día

▼ Release Planning

Se decide qué US, épicas o temas se espera incluir en una entrega.

El plan **se actualiza** mientras avanza el proyecto. Se apoya en prioridades, estimaciones y velocidad/capacidad del equipo.

No se congela el backlog al inicio del release.

▼ Planificación de Iteración / Sprint

Al comenzar la iteración se selecciona trabajo de mayor prioridad considerando tiempo, capacidad y objetivos del Sprint.

▼ Planificación diaria

Coordina el trabajo cotidiano y permite ajustar próximas actividades según el estado real del Sprint.

▼ 9. Scrum: marco, empirismo y roles

Scrum es un **framework ágil, iterativo-incremental y empírico**. Se apoya en transparencia, inspección y adaptación.

▼ Product Owner

- maximiza el valor del producto;
- ordena y prioriza el Product Backlog;
- define **qué** se necesita y qué tiene mayor valor;
- no decide **cómo** implementar técnicamente el trabajo.

▼ Developers

- construyen el incremento;
- se autogestionan;
- deciden **cómo** llevar adelante el trabajo;
- estiman su propio trabajo.

▼ Scrum Master

- facilita Scrum;
- ayuda a remover impedimentos y mejorar el proceso;
- promueve empirismo y mejora continua;
- **no decide cuestiones técnicas ni de negocio por el equipo.**

Machete: PO = qué/prioridad. Developers = cómo. Scrum Master = facilita.



10. Scrum: Sprint, eventos, artefactos y compromisos

▼ Sprint / Timebox

El Sprint es una iteración de duración establecida. Durante él:

- no se deben hacer cambios que pongan en peligro el Sprint Goal;
- la calidad no disminuye;
- el backlog puede refinarse;
- el alcance puede **clarificarse y negociarse** con el PO según se aprende.

Matiz importante: decir “el alcance jamás cambia durante un Sprint” es una simplificación. Lo que no debe ponerse en riesgo es el **Sprint Goal**.

▼ Eventos

- **Sprint Planning:** define por qué el Sprint es valioso, qué puede hacerse y cómo se realizará.
- **Daily Scrum:** inspecciona progreso hacia Sprint Goal y adapta el trabajo próximo.
- **Sprint Review:** revisa resultados con stakeholders y determina futuras adaptaciones.
- **Sprint Retrospective:** busca mejorar calidad y eficiencia del equipo y proceso.
- **Refinamiento:** mejora y aclara ítems del Product Backlog para trabajo futuro.

▼ Artefactos y compromisos

- **Product Backlog → Product Goal.**
- **Sprint Backlog → Sprint Goal.**
- **Incremento → Definition of Done.**

Esta asociación aparece literalmente en preguntas: conviene memorizarla.



11. Errores Scrum frecuentes que aparecieron en parciales

- Product Backlog desactualizado.
- Scrum Master definiendo el Sprint Backlog por los Developers.
- Incremento que no cumple DoD.
- Creer que “hacer reuniones” alcanza para cumplir Scrum.
- PO poco disponible, generando bloqueos.
- Historias entrando al Sprint sin suficiente refinamiento.
- Sprint Review donde el PO dice “no era lo que esperaba”: puede indicar criterios de aceptación insuficientes, problemas de DoR/DoD y falta de transparencia.



UNIDAD 3 — Software Configuration Management (SCM)

▼ 1. Qué es SCM y cuál es su objetivo

SCM — Gestión de Configuración de Software — es una disciplina de soporte transversal al proyecto y al ciclo de vida del producto.

Su objetivo central es **establecer y mantener la integridad del software**.

Para eso se ocupa de:

- identificar ítems y configuraciones;
- controlar cambios;
- registrar el estado de implementación;
- mantener trazabilidad;
- verificar correspondencia con requerimientos.



2. Ítem de Configuración, configuración, versión y variante

▼ Ítem de Configuración — IC/SCI

Es cualquier artefacto del producto o proyecto cuyo estado/evolución necesitamos controlar porque puede cambiar o debe compartirse.

Ejemplos: requerimientos, documentos, código fuente, ejecutables, planes, arquitectura, casos de prueba, etc.

▼ Configuración

Conjunto de ítems de configuración con sus versiones en un momento determinado.

▼ Versión

Forma particular de un artefacto en un instante o contexto determinado.

▼ Variante

Versión/configuración alternativa que evoluciona por separado. Puede representar diferentes plataformas, opciones funcionales o líneas paralelas del producto.

▼ 3. Línea Base — Baseline

Una baseline es una **configuración revisada formalmente y acordada**, considerada estable bajo determinados criterios.

Sirve para:

- tener un punto de referencia;
- reproducir el entorno/configuración de un momento anterior;
- servir como base para desarrollos posteriores;
- controlar formalmente cambios posteriores.

No se cambia libremente. Una vez establecida, los cambios pasan por un procedimiento formal.

No confundir: baseline **no es lo mismo que versión del producto ni rama principal.**

▼ 4. Repositorio y control de versiones

El repositorio contiene los ítems de configuración y conserva su historial, atributos y relaciones.

Conceptos prácticos:

- **Check out / extracción:** traer una copia desde el repositorio al entorno local.
- **Commit:** registrar localmente un conjunto de cambios.
- **Check in / devolución:** enviar cambios nuevamente al repositorio.

El control de versiones permite conocer **qué cambió, cuándo y sobre qué versión.**

▼ 5. Ramas y Merge

Las ramas permiten bifurcar el desarrollo para trabajar en paralelo o experimentar.

- una rama puede integrarse o descartarse;
- **merge** es la operación que integra cambios;
- pueden aparecer conflictos que deben resolverse;
- una estrategia de ramas clara mejora control y trazabilidad.

▼ Problemas típicos de SCM en ramas

Si al integrar nadie sabe qué terminó dentro del ejecutable entregado, pueden existir:

- identificación deficiente de IC;
- control de versiones pobre;
- trazabilidad insuficiente fuente ↔ ejecutable;
- falta de verificación/auditoría de la configuración entregada.

▼ 6. Planificación de SCM

Un plan de configuración debería definir, entre otras cosas:

- reglas de nombrado de IC;
- herramientas utilizadas;
- roles y participantes;
- procedimiento formal de cambios;
- formularios y registros;
- procesos y periodicidad de auditoría.

Trampa típica: realizar auditorías únicamente al final. Si hay entregas parciales, conviene planificarlas en puntos de control durante el proyecto y dejar evidencia de sus resultados.

▼ 7. Control formal de cambios y CCC

CCC = Comité de Control de Cambios.

Cuando un cambio afecta ítems que pertenecen a una línea base, no debería implementarse informalmente.

Flujo mental:

Solicitud → evaluación de impacto → decisión → aprobación/rechazo → orden/registro → implementación.

La evaluación puede considerar:

- impacto técnico;
- arquitectura;
- costos;
- cronograma;
- efectos colaterales;
- impacto general del proyecto.

Cliente pide un cambio ≠ cambio automáticamente aprobado.

Si el cambio afecta arquitectura, consultar especialistas del área puede formar parte de una correcta evaluación del CCC.

▼ 8. Auditorías: FCA y PCA

▼ FCA — Functional Configuration Audit

Se orienta a comprobar que la funcionalidad/performance implementada **corresponda con lo especificado y aprobado**.

En las preguntas revisadas se relaciona con **Verificación**.

▼ PCA — Physical Configuration Audit

Se orienta a comprobar la existencia, identificación y consistencia física/documental de los elementos de configuración que conforman el producto.

▼ Verificación vs. Validación

- **Verificación:** "¿Estamos construyendo correctamente lo especificado?"
- **Validación:** "¿Estamos construyendo el producto correcto para la necesidad del usuario?"

Machete del parcial: si dice *"la funcionalidad no cumple con los requerimientos aprobados"* → **FCA + Verificación**.

▼ 9. Integridad y trazabilidad

La integridad del producto implica poder saber qué configuración corresponde a qué estado, mantener el origen de los cambios y rastrear los componentes durante el ciclo de vida.

Problemas que SCM intenta prevenir:

- pérdida de componentes;
- pérdida de cambios;
- desincronización fuente/objeto/ejecutable;
- regresión de fallas;
- doble mantenimiento;
- cambios superpuestos;
- cambios no válidos.

▼ 10. Machete final de SCM

- **IC:** artefacto que quiero controlar.
- **Versión:** estado del IC en un momento.
- **Variante:** alternativa que evoluciona por separado.
- **Baseline:** configuración estable, revisada y acordada.
- **Rama:** bifurcación del desarrollo.
- **Merge:** integración de ramas.
- **CCC:** decide sobre cambios formales luego de evaluar impacto.
- **FCA:** funcionalidad vs. especificación.
- **PCA:** configuración física/documental.
- **Verificación:** construir según lo especificado.
- **Validación:** construir lo que realmente necesita el usuario.



REPASO RÁPIDO — asociaciones que más aparecieron en preguntas

- **Secuencial** → certeza / plan.
- **Iterativo-incremental** → cambio / feedback / entregas parciales.
- **Recursivo** → riesgo.
- **Empírico** → transparencia + inspección + adaptación.
- **No Silver Bullet** → una herramienta ayuda, pero no elimina complejidad esencial.
- **MVP** → aprendizaje validado.
- **MMF/MMP** → valor comercial.
- **US grande** → dividir en US verticales.
- **DoR** → lista para empezar.
- **DoD** → terminada.
- **Story Points** → tamaño relativo, NO horas.

- **Velocidad** → se calcula con US terminadas.
- **Estimación** → no es compromiso.
- **PO** → qué y prioridad.
- **Developers** → cómo.
- **Scrum Master** → facilita.
- **Product Backlog** → Product Goal.
- **Sprint Backlog** → Sprint Goal.
- **Incremento** → DoD.
- **Agile** → adaptar backlog.
- **Baseline** → control formal de cambios.
- **CCC** → evaluar y decidir cambios.
- **Requerimientos aprobados vs. implementación** → FCA + Verificación.